

Incident Memory

An MCP server with long-term incident memory, backed by Amazon DynamoDB, deployed on AWS EC2

Python · FastMCP

DynamoDB

AWS EC2

Docker · nginx · systemd

Complete Project Guide — architecture, how everything works, DynamoDB data model, MCP implementation, AWS credentials & IAM, step-by-step EC2 deployment, verification and resume notes.

Version 1.0 · September 2026

Table of Contents

1. Project overview & why it matters for your resume
2. Architecture — how the whole thing works end-to-end
3. MCP fundamentals you need
4. DynamoDB fundamentals & the data model
5. Project structure
6. Phase 1 — Local development (MCP server + DynamoDB Local)
7. Phase 2 — Implementing the tools (code walkthrough)
8. Phase 3 — AWS setup: account, IAM, credentials, the real table
9. Phase 4 — Deploying on EC2 (step by step)
10. Phase 5 — Connecting Claude to your server
11. Verification checklist
12. Costs, cleanup & security checklist
13. Resume bullets & interview questions

1. Project Overview

Incident Memory is an on-call engineer's assistant. Every production incident (alert, outage, root cause, fix) is stored in DynamoDB. An **MCP server** exposes tools so an AI client such as Claude can *search, record and reason over* that history. Example conversation:

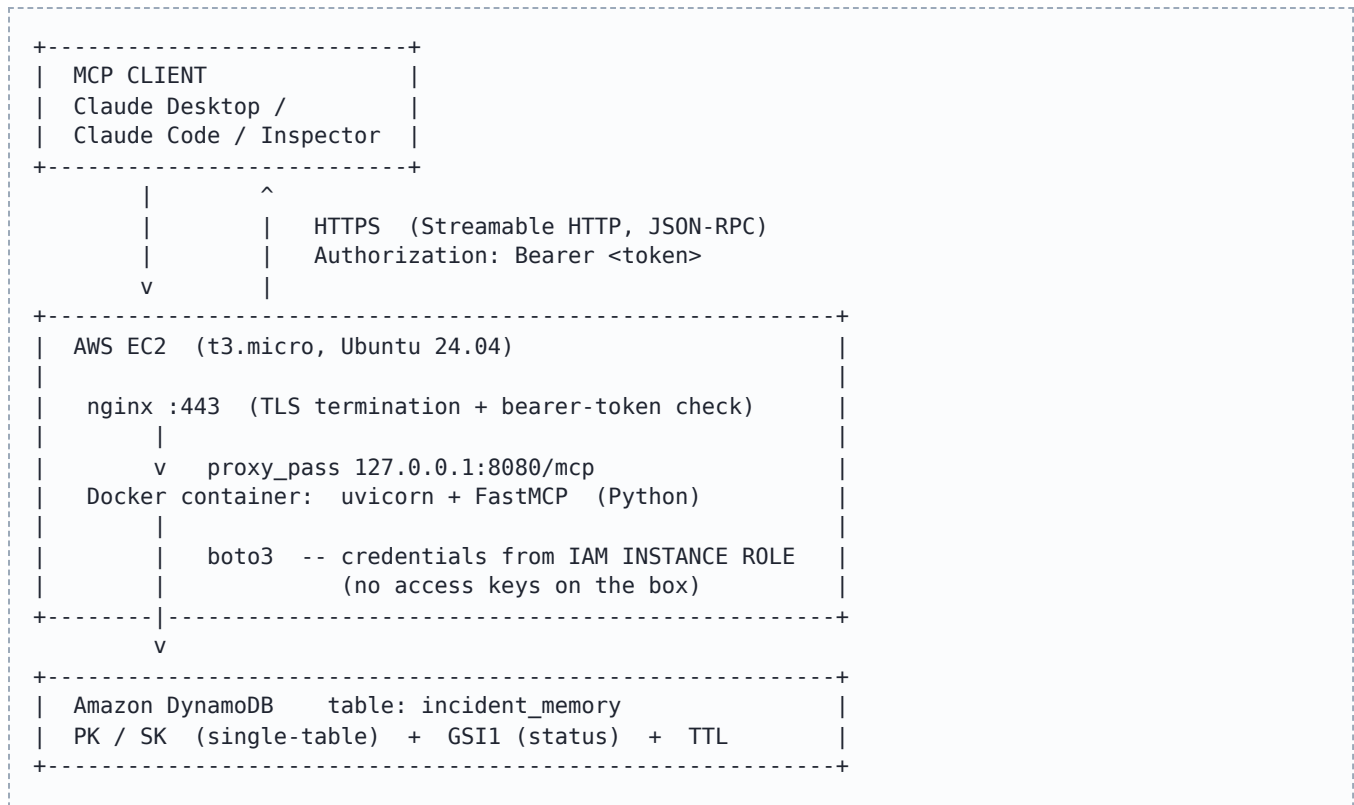
User: "checkout-service is returning 503s again."

Claude: calls `find_similar_incidents(service="checkout-service", keyword="503")` → finds 2 past incidents → "This happened on 14 June and 2 Aug. Both times the root cause was a Redis connection pool exhaustion; the fix was restarting the pod and raising `maxclients`. Want me to open a new incident?"

What you will learn (and can honestly put on your resume)

Technology	Concrete skills demonstrated
MCP (Model Context Protocol)	Building a server with tools, resources and prompts; stdio vs Streamable HTTP transports; schema design for LLM tool-calling; authenticating a remote MCP server; testing with MCP Inspector.
DynamoDB	Single-table design; partition/sort key modelling; Global Secondary Index; conditional writes (optimistic locking); atomic counters; TTL; on-demand billing; local development with DynamoDB Local; mocking with moto.
AWS / EC2 / DevOps	IAM users vs instance roles; least-privilege policies; security groups; Docker; systemd; nginx reverse proxy; HTTPS; CloudWatch logs.

2. Architecture — How It Works End-to-End



Request flow, step by step

1. The user types a message in Claude. Claude decides a tool is needed and sends a JSON-RPC `tools/call` request to `https://your-host/mcp`.
2. **nginx** terminates TLS, checks the `Authorization: Bearer` header, and proxies the request to the container on port 8080.
3. **MCPServer / FastMCP** (Python) parses the request, validates the arguments against the tool's schema and calls your Python function.
4. Your function uses **boto3** to Query / PutItem / UpdateItem on DynamoDB. boto3 automatically gets temporary credentials from the **EC2 instance role** — no access keys are stored on the server.
5. The result is returned as JSON → FastMCP → nginx → Claude, which writes the natural-language answer.

Two environments

	Local (your laptop)	Production (EC2)
MCP transport	stdio (Claude launches the Python process)	Streamable HTTP over HTTPS
DynamoDB	DynamoDB Local in Docker (<code>http://localhost:8002</code>)	Real DynamoDB in <code>eu-central-1</code> (or your region)
Credentials	Dummy keys for DynamoDB Local; named AWS profile for the real table	IAM instance role — zero keys
Auth	none	Bearer token enforced by nginx

3. MCP Fundamentals You Need

MCP (Model Context Protocol) is an open standard that lets an AI application (the *client/host*) talk to external systems through a *server*. The server advertises three kinds of capabilities:

Primitive	Who triggers it	Purpose	In this project
Tool	The model	Executes an action / query, returns data	<code>record_incident</code> , <code>find_similar_incidents</code> , ...
Resource	The application	Read-only data addressed by URI	<code>incident://checkout-service/latest</code>
Prompt	The user	Reusable prompt template	<code>triage_alert</code>

Transports

- **stdio** — the client starts your server as a subprocess and talks over stdin/stdout. Perfect for local development.
- **Streamable HTTP** — the server is a web service with a single `/mcp` endpoint (POST for requests, optional SSE stream for server→client messages). This is what you deploy on EC2.

Why tool design matters

The model only sees the tool *name*, *description* and *JSON schema*. Good descriptions = good tool selection. FastMCP generates the schema from your Python type hints and the docstring, so write docstrings as if explaining to a new colleague.

Testing tool

MCP Inspector is a browser UI for calling your server manually: `npx @modelcontextprotocol/inspector`. Use it before connecting Claude.

4. DynamoDB Fundamentals & the Data Model

Core concepts (the ones interviewers ask about)

Concept	Meaning
Partition key (PK)	Decides which physical partition stores the item. All queries must specify it (unless using Scan — avoid).
Sort key (SK)	Orders items within a partition; supports <code>begins_with</code> , <code>between</code> , <code>></code> , <code><</code> .
Single-table design	Store different entity types in one table using generic <code>PK / SK</code> attributes with prefixes like <code>SERVICE#</code> , <code>INCIDENT#</code> .
GSI (Global Secondary Index)	A copy of the table with a different key, enabling a second access pattern (e.g. "all open incidents across services").
Condition expression	Make a write succeed only if a condition holds (e.g. status is still "open"). Prevents lost updates.
Atomic counter	<code>UpdateItem ... SET count = count + :one</code> (or <code>ADD</code>) — safe increments without read-modify-write.
TTL	Mark an epoch-seconds attribute; DynamoDB deletes expired items for free.
On-demand billing	Pay per request — ideal for portfolio projects (essentially \$0).

Access patterns first, then keys

In DynamoDB you design the table *from the queries*, not from the entities. Our access patterns:

1. Get all incidents for a service, newest first
2. Search incidents of a service by keyword
3. Get one incident by ID
4. List all *open* incidents across all services
5. Resolve an incident (only if still open)
6. Add a runbook note to a service that expires after N days
7. Get per-service stats (incident count, last incident)

Table: `incident_memory`

Entity	PK	SK	GSI1PK	GSI1SK	Other attributes
Incident	<code>SERVICE#checkout</code>	<code>INCIDENT#2026-09-12T10:15:00Z#01J...</code>	<code>STATUS#open</code>	<code>2026-09-12T10:15:00Z</code>	incident_id, severity, summary, root_cause, fix, status, created_at, resolved_at
Runbook note	<code>SERVICE#checkout</code>	<code>NOTE#2026-09-12T11:00:00Z</code>	—	—	note, author, expires_at (TTL)
Service meta	<code>SERVICE#checkout</code>	<code>META</code>	—	—	incident_count, last_incident_at

Access pattern	DynamoDB operation
1. Incidents for a service, newest first	Query <code>PK = SERVICE#x AND begins_with(SK, "INCIDENT#")</code> , <code>ScanIndexForward=False</code>
2. Keyword search	Same query + <code>FilterExpression contains(summary, :kw) OR contains(root_cause, :kw)</code>
3. Get by ID	Incident ID encodes service + SK → <code>GetItem</code>
4. All open incidents	Query <code>GSI1: GSI1PK = STATUS#open</code> , sorted by <code>GSI1SK</code> (time)
5. Resolve	<code>UpdateItem SET status=:resolved, GSI1PK=:closed ...</code> <code>ConditionExpression status = :open</code>
6. Runbook note with expiry	<code>PutItem</code> with <code>expires_at = now + N days (epoch seconds)</code> ; TTL enabled on <code>expires_at</code>
7. Service stats	<code>UpdateItem ADD incident_count :one</code> on the <code>META</code> item; <code>GetItem META</code> to read

Why a GSI for "open incidents"? Because the main table's PK is the service, so "all open incidents across services" would require a Scan. The GSI re-keys the data by status so it becomes a cheap Query.

5. Project Structure

```
incident_memory/
├── src/incident_memory/
│   ├── __init__.py
│   ├── server.py           # FastMCP app: tools, resource, prompt
│   ├── db.py              # boto3 table client, key builders, all DynamoDB calls
│   ├── models.py          # pydantic models (Incident, Note, ServiceStats)
│   └── config.py          # settings from env vars
├── tests/
│   ├── conftest.py        # moto-mocked DynamoDB table fixture
│   └── test_db.py
├── infra/
│   ├── create_table.py    # creates table + GSI + enables TTL
│   └── iam_policy.json    # least-privilege policy for the EC2 role
├── deploy/
│   ├── Dockerfile
│   ├── docker-compose.local.yml # app + amazon/dynamodb-local
│   ├── incident-memory.service # systemd unit
│   └── nginx.conf
├── docs/
│   └── DESIGN.md          # access patterns + key design (interview talking points)
├── .env.example
├── pyproject.toml
└── README.md
```

Environment variables (config.py)

Variable	Local	EC2
TABLE_NAME	incident_memory	incident_memory
AWS_REGION	eu-central-1	eu-central-1
DDB_ENDPOINT	http://localhost:8002 (DynamoDB Local)	unset → real AWS
MCP_TRANSPORT	stdio	streamable-http
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	local / local (DynamoDB Local accepts anything)	never set — instance role

6. Phase 1 — Local Development

6.1 Prerequisites

- Python 3.12+, uv (or pip), Docker, Node.js (for MCP Inspector), AWS CLI v2

6.2 Create the project

```
mkdir -p incident_memory && cd incident_memory
uv init --package . # or: python -m venv .venv && source .venv/bin/activate
uv add "mcp[cli]>=2.2" boto3 pydantic python-ulid
uv add --dev pytest moto[dynamodb]
```

6.3 Run DynamoDB Local

```
# deploy/docker-compose.local.yml
services:
  dynamodb:
    image: amazon/dynamodb-local:latest
    command: "-jar DynamoDBLocal.jar -sharedDb -dbPath /data"
    ports: ["8002:8000"] # host 8002 -> container 8000
    volumes: ["ddb-data:/data"]
volumes:
  ddb-data:
```

```
docker compose -f deploy/docker-compose.local.yml up -d
export AWS_ACCESS_KEY_ID=local AWS_SECRET_ACCESS_KEY=local AWS_REGION=eu-central-1
export DDB_ENDPOINT=http://localhost:8002 TABLE_NAME=incident_memory
python infra/create_table.py # creates table, GSI, TTL
aws dynamodb list-tables --endpoint-url http://localhost:8002
```

6.4 Table creation script

```
# infra/create_table.py
import os, boto3

ddb = boto3.client("dynamodb",
                    region_name=os.getenv("AWS_REGION", "eu-central-1"),
                    endpoint_url=os.getenv("DDB_ENDPOINT")) # None -> real AWS
table = os.getenv("TABLE_NAME", "incident_memory")

ddb.create_table(
    TableName=table,
    BillingMode="PAY_PER_REQUEST",
    AttributeDefinitions=[
        {"AttributeName": "PK", "AttributeType": "S"},
        {"AttributeName": "SK", "AttributeType": "S"},
        {"AttributeName": "GSI1PK", "AttributeType": "S"},
        {"AttributeName": "GSI1SK", "AttributeType": "S"},
    ],
    KeySchema=[{"AttributeName": "PK", "KeyType": "HASH"},
                {"AttributeName": "SK", "KeyType": "RANGE"}],
    GlobalSecondaryIndexes=[{
        "IndexName": "GSI1",
        "KeySchema": [{"AttributeName": "GSI1PK", "KeyType": "HASH"},
                      {"AttributeName": "GSI1SK", "KeyType": "RANGE"}],
        "Projection": {"ProjectionType": "ALL"},
    }],
)
ddb.get_waiter("table_exists").wait(TableName=table)
ddb.update_time_to_live(TableName=table,
    TimeToLiveSpecification={"Enabled": True, "AttributeName": "expires_at"})
print("created", table)
```

7. Phase 2 — Implementing the Tools

7.1 db.py — the DynamoDB layer

```
# src/incident_memory/db.py
import os, time
from datetime import datetime, timezone, timedelta
import boto3
from boto3.dynamodb.conditions import Key, Attr
from botocore.exceptions import ClientError
from ulid import ULID

_res = boto3.resource("dynamodb",
                      region_name=os.getenv("AWS_REGION", "eu-central-1"),
                      endpoint_url=os.getenv("DDB_ENDPOINT"))
table = _res.Table(os.getenv("TABLE_NAME", "incident_memory"))

def _now() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

def record_incident(service, severity, summary, root_cause="", fix=""):
    ts, uid = _now(), str(ULID())
    sk = f"INCIDENT#{ts}#{uid}"
    item = {"PK": f"SERVICE#{service}", "SK": sk,
            "GSI1PK": "STATUS#open", "GSI1SK": ts,
            "incident_id": f"{service}|{sk}", "service": service,
            "severity": severity, "summary": summary, "root_cause": root_cause,
            "fix": fix, "status": "open", "created_at": ts}
    table.put_item(Item=item)
    # atomic counter on the META item
    table.update_item(
        Key={"PK": f"SERVICE#{service}", "SK": "META"},
        UpdateExpression="ADD incident_count :one SET last_incident_at = :ts",
        ExpressionAttributeValues={":one": 1, ":ts": ts})
    return item

def find_similar(service, keyword=None, limit=5):
    kwargs = dict(
        KeyConditionExpression=Key("PK").eq(f"SERVICE#{service}") &
        Key("SK").begins_with("INCIDENT#"),
        ScanIndexForward=False, Limit=50)
    if keyword:
        kwargs["FilterExpression"] = (Attr("summary").contains(keyword) |
                                     Attr("root_cause").contains(keyword))
    return table.query(**kwargs)["Items"][:limit]

def get_incident(incident_id):
    service, sk = incident_id.split("|", 1)
    return table.get_item(Key={"PK": f"SERVICE#{service}", "SK": sk}).get("Item")

def resolve_incident(incident_id, root_cause, fix):
    service, sk = incident_id.split("|", 1)
    try:
        return table.update_item(
            Key={"PK": f"SERVICE#{service}", "SK": sk},
            UpdateExpression="SET #s=:resolved, GSI1PK=:g, root_cause=:rc, fix=:fx,
resolved_at=:ts",
```

```

        ConditionExpression="#s = :open",          # optimistic locking
        ExpressionAttributeNames={"#s": "status"},
        ExpressionAttributeValues={":resolved": "resolved", ":open": "open",
                                    ":g": "STATUS#resolved", ":rc": root_cause,
                                    ":fx": fix, ":ts": _now()},
        ReturnValues="ALL_NEW")["Attributes"]
except ClientError as e:
    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
        raise ValueError("Incident is not open (already resolved?)")
    raise

def list_open(limit=20):
    return table.query(IndexName="GSI1",
        KeyConditionExpression=Key("GSI1PK").eq("STATUS#open"),
        ScanIndexForward=False, Limit=limit)["Items"]

def add_note(service, note, author, ttl_days=30):
    ts = _now()
    item = {"PK": f"SERVICE#{service}", "SK": f"NOTE#{ts}", "note": note,
            "author": author, "created_at": ts,
            "expires_at": int(time.time() + ttl_days * 86400)}    # TTL
    table.put_item(Item=item)
    return item

def service_stats(service):
    return table.get_item(Key={"PK": f"SERVICE#{service}", "SK": "META"}).get("Item", {})

```

7.2 server.py — the MCP layer

```
# src/incident_memory/server.py
import os
from typing import Literal
from mcp.server.mcpserver import MCPServer # mcp 2.x (was FastMCP in 1.x)
from . import db

mcp = MCPServer("incident-memory")

@mcp.tool()
def record_incident(service: str,
                    severity: Literal["sev1", "sev2", "sev3", "sev4"],
                    summary: str, root_cause: str = "", fix: str = "") -> dict:
    """Record a new production incident for a service. Use when a user reports
    an outage, alert or degradation. Returns the created incident including its
    incident_id."""
    return db.record_incident(service, severity, summary, root_cause, fix)

@mcp.tool()
def find_similar_incidents(service: str, keyword: str | None = None, limit: int = 5) ->
list[dict]:
    """Search past incidents of a service, newest first. Optionally filter by a keyword
    found in the summary or root cause (e.g. '503', 'redis', 'timeout')."""
    return db.find_similar(service, keyword, limit)

@mcp.tool()
def get_incident(incident_id: str) -> dict | None:
    """Fetch one incident by its incident_id."""
    return db.get_incident(incident_id)

@mcp.tool()
def update_incident_status(incident_id: str, root_cause: str, fix: str) -> dict:
    """Mark an open incident as resolved, recording the root cause and the fix.
    Fails if the incident is already resolved."""
    return db.resolve_incident(incident_id, root_cause, fix)

@mcp.tool()
def list_open_incidents(limit: int = 20) -> list[dict]:
    """List currently open incidents across all services, newest first."""
    return db.list_open(limit)

@mcp.tool()
def add_runbook_note(service: str, note: str, author: str = "claude", ttl_days: int = 30) ->
dict:
    """Attach a temporary runbook note to a service. Expires automatically after ttl_days."""
    return db.add_note(service, note, author, ttl_days)

@mcp.tool()
def service_stats(service: str) -> dict:
    """Return incident_count and last_incident_at for a service."""
    return db.service_stats(service)

@mcp.resource("incident://{service}/latest")
def latest_incident(service: str) -> str:
    """The most recent incident of a service, as text."""
    items = db.find_similar(service, limit=1)
    return str(items[0]) if items else f"No incidents recorded for {service}."

@mcp.prompt()
```

```
def triage_alert(service: str, alert_text: str) -> str:
    """Guide the model through triaging a new alert using past incidents."""
    return (f"An alert fired for {service}: {alert_text}\n"
            "1. Call find_similar_incidents to check history.\n"
            "2. Summarise likely root cause and past fixes.\n"
            "3. Ask whether to record_incident.")

def main():
    if os.getenv("MCP_TRANSPORT") == "streamable-http":
        mcp.run(transport="streamable-http", host="0.0.0.0", port=8080) # EC2 / Docker
    else:
        mcp.run(transport="stdio") # local

if __name__ == "__main__":
    main()
```

Add to `pyproject.toml`: `[project.scripts] incident-memory = "incident_memory.server:main"`

7.3 Test locally

```
# Inspector (stdio)
npx @modelcontextprotocol/inspector uv run incident-memory

# Claude Code (stdio)
claude mcp add incident-memory -e DDB_ENDPOINT=http://localhost:8002 \
  -e AWS_ACCESS_KEY_ID=local -e AWS_SECRET_ACCESS_KEY=local -e AWS_REGION=eu-central-1 \
  -- uv run --directory /path/to/incident_memory incident-memory

# Unit tests with moto
uv run pytest
```

```
# tests/conftest.py (moto)
import os, boto3, pytest
from moto import mock_aws

@pytest.fixture(autouse=True)
def ddb_table(monkeypatch):
    monkeypatch.setenv("AWS_ACCESS_KEY_ID", "test");
    monkeypatch.setenv("AWS_SECRET_ACCESS_KEY", "test")
    monkeypatch.setenv("AWS_REGION", "eu-central-1"); monkeypatch.delenv("DDB_ENDPOINT",
    raising=False)
    with mock_aws():
        import importlib, infra.create_table # runs create_table against moto
        from incident_memory import db
        importlib.reload(db)
        yield db
```

8. Phase 3 — AWS Setup: Account, IAM & Credentials

Golden rules

1. Never use the root account for daily work — create an IAM user/identity and enable MFA on root.
2. Never put access keys on the EC2 instance, in code, in Docker images or in git. EC2 uses an **instance role**.
3. Grant the minimum permissions needed (one table, a handful of actions).

8.1 Credentials you will have — and where each is used

Credential	Used where	How to get it	Notes
Root account (email + password + MFA)	Only to create the first IAM user and billing alerts	AWS sign-up	Lock it away. Turn on MFA.
IAM user for development (e.g. <code>dev-admin</code>) — Access Key ID + Secret Access Key	Your laptop: AWS CLI, <code>create_table.py</code> , running server locally against the real table	IAM → Users → Create user → attach <code>PowerUserAccess</code> (or <code>AdministratorAccess</code> for learning) → Security credentials → Create access key (CLI)	Stored only in <code>~/.aws/credentials</code> via <code>aws configure --profile incident</code> . Rotate/delete when done.
IAM role for EC2 (<code>IncidentMemoryEC2Role</code>)	On the instance — boto3 gets temporary creds automatically	IAM → Roles → Create role → Trusted entity: <i>AWS service</i> → <i>EC2</i> → attach the custom policy below	No keys, auto-rotated every few hours by AWS.
EC2 key pair (<code>.pem</code> file)	SSH from laptop to the instance	Created when launching the instance	<code>chmod 400</code> ; never share.
MCP bearer token	Sent by Claude in <code>Authorization</code> header; checked by nginx	<code>openssl rand -hex 32</code>	Your own secret; store in Claude's MCP config and in nginx.
TLS certificate	nginx HTTPS	Let's Encrypt via certbot (needs a domain) or self-signed	Free.

8.2 Configure the AWS CLI on your laptop

```
aws configure --profile incident
# AWS Access Key ID:      AKIA.....
# AWS Secret Access Key: .....
# Default region name:    eu-central-1
# Default output format: json
export AWS_PROFILE=incident
aws sts get-caller-identity      # verify who you are
```

8.3 Create the real table

```
unset DDB_ENDPOINT              # important: point at real AWS
AWS_PROFILE=incident python infra/create_table.py
aws dynamodb describe-table --table-name incident_memory --query 'Table.TableStatus'
```

8.4 Least-privilege IAM policy for the EC2 role

```
// infra/iam_policy.json - replace ACCOUNT_ID and region
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "IncidentMemoryTableAccess",
    "Effect": "Allow",
    "Action": [
      "dynamodb:GetItem", "dynamodb:PutItem", "dynamodb:UpdateItem",
      "dynamodb:Query", "dynamodb:DescribeTable"
    ],
    "Resource": [
      "arn:aws:dynamodb:eu-central-1:ACCOUNT_ID:table/incident_memory",
      "arn:aws:dynamodb:eu-central-1:ACCOUNT_ID:table/incident_memory/index/GSI1"
    ]
  }]
}
```

```
aws iam create-policy --policy-name IncidentMemoryDynamoDB --policy-document
file://infra/iam_policy.json
aws iam create-role --role-name IncidentMemoryEC2Role --assume-role-policy-document '{
  "Version": "2012-10-17", "Statement": [{"Effect": "Allow",
  "Principal": {"Service": "ec2.amazonaws.com"}, "Action": "sts:AssumeRole"}]}'
aws iam attach-role-policy --role-name IncidentMemoryEC2Role \
  --policy-arn arn:aws:iam::ACCOUNT_ID:policy/IncidentMemoryDynamoDB
aws iam create-instance-profile --instance-profile-name IncidentMemoryEC2Role
aws iam add-role-to-instance-profile --instance-profile-name IncidentMemoryEC2Role --role-name
IncidentMemoryEC2Role
```

Note: no `Scan` and no `DeleteItem` — the app never needs them, so the role can't do them. That is a talking point in interviews.

9. Phase 4 — Deploying on EC2

9.1 Launch the instance (AWS Console → EC2 → Launch instance)

Setting	Value
Name	incident-memory
AMI	Ubuntu Server 24.04 LTS (x86_64)
Instance type	t3.micro (or t2.micro — free tier eligible)
Key pair	Create new → incident-memory-key → download .pem
Network / Security group	Create new: allow SSH (22) from <i>My IP</i> only; HTTPS (443) from anywhere; HTTP (80) from anywhere (only needed for certbot). <u>Do not</u> open 8080.
Advanced → IAM instance profile	IncidentMemoryEC2Role ← this is how the server gets DynamoDB access
Storage	8–10 GB gp3
(Optional) Elastic IP	Allocate & associate so the public IP doesn't change on restart

9.2 Connect and install Docker

```
chmod 400 ~/Downloads/incident-memory-key.pem
ssh -i ~/Downloads/incident-memory-key.pem ubuntu@<EC2_PUBLIC_IP>

sudo apt update && sudo apt install -y ca-certificates curl git nginx
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu && newgrp docker

# sanity check: the instance role works, no keys needed
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

9.3 Dockerfile

```
# deploy/Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY pyproject.toml uv.lock README.md ./
COPY src ./src
RUN pip install --no-cache-dir uv && uv pip install --system .
ENV MCP_TRANSPORT=streamable-http TABLE_NAME=incident_memory AWS_REGION=eu-central-1
EXPOSE 8080
CMD ["incident-memory"]
```

9.4 Build and run

```
git clone https://github.com/<you>/incident_memory.git && cd incident_memory
docker build -t incident-memory -f deploy/Dockerfile .
docker run -d --name incident-memory --restart unless-stopped \
  -p 127.0.0.1:8080:8080 \
  -e MCP_TRANSPORT=streamable-http -e TABLE_NAME=incident_memory -e AWS_REGION=eu-central-1 \
  --log-driver awslogs --log-opt awslogs-region=eu-central-1 \
  --log-opt awslogs-group=/incident-memory --log-opt awslogs-create-group=true \
  incident-memory
docker logs -f incident-memory      # should show uvicorn listening on 0.0.0.0:8080
```

Binding to `127.0.0.1:8080` means only nginx on the same box can reach the app. For the awslogs driver, add `logs:CreateLogGroup`, `logs:CreateLogStream`, `logs:PutLogEvents` to the role policy, or drop the `--log-*` flags.

9.5 systemd unit (auto-start on reboot)

```
# /etc/systemd/system/incident-memory.service
[Unit]
Description=Incident Memory MCP server
After=docker.service
Requires=docker.service

[Service]
Restart=always
ExecStart=/usr/bin/docker start -a incident-memory
ExecStop=/usr/bin/docker stop incident-memory

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now incident-memory
```

9.6 nginx reverse proxy with bearer-token auth

```
# /etc/nginx/sites-available/incident-memory
server {
    listen 443 ssl;
    server_name mcp.yourdomain.com;          # or the EC2 public DNS name

    ssl_certificate      /etc/letsencrypt/live/mcp.yourdomain.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/mcp.yourdomain.com/privkey.pem;

    location /mcp {
        if ($http_authorization != "Bearer REPLACE_WITH_openssl_rand_hex_32") {
            return 401;
        }
        proxy_pass http://127.0.0.1:8080/mcp;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header Connection "";
        proxy_buffering off;                  # required for SSE streaming
        proxy_read_timeout 3600s;
    }
    location / { return 404; }
}
server { listen 80; server_name mcp.yourdomain.com; return 301 https://$host$request_uri; }
```

```
sudo ln -s /etc/nginx/sites-available/incident-memory /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

9.7 HTTPS certificate

Option A — real domain (recommended): point an A record (`mcp.yourdomain.com`) at the Elastic IP, then:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d mcp.yourdomain.com      # auto-edits nginx, auto-renews
```

Option B — no domain (self-signed, for testing only):

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/ssl/private/mcp.key -out /etc/ssl/certs/mcp.crt -subj "/CN=ec2-
xx.compute.amazonaws.com"
# then set ssl_certificate / ssl_certificate_key to these paths
```

9.8 Smoke test from your laptop

```
TOKEN=<your token>
curl -s https://mcp.yourdomain.com/mcp -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" -H "Accept: application/json, text/event-stream" \
    -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-06-18",
        "capabilities":{},"clientInfo":{"name":"curl","version":"0"}}}'
# → JSON with serverInfo "incident-memory". Without the header → 401.
```

10. Phase 5 — Connecting Claude to Your Server

Claude Code

```
claude mcp add --transport http incident-memory https://mcp.yourdomain.com/mcp \
  --header "Authorization: Bearer <token>"
claude          # then: "Has checkout-service had 503 errors before?"
```

Claude Desktop

Settings → Connectors → *Add custom connector* → URL `https://mcp.yourdomain.com/mcp`. If the UI does not allow a custom header, bridge it via `mcp-remote` in `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "incident-memory": {
      "command": "npx",
      "args": ["-y", "mcp-remote", "https://mcp.yourdomain.com/mcp",
        "--header", "Authorization: Bearer <token>"]
    }
  }
}
```

MCP Inspector (remote)

```
npx @modelcontextprotocol/inspector
# Transport: Streamable HTTP → URL https://mcp.yourdomain.com/mcp → add Authorization header →
Connect
```

Demo script for your README GIF

1. "Record a sev2 incident for checkout-service: 503 errors on /pay, started 10:15."
2. "Any similar incidents for checkout-service mentioning 503?"
3. "Resolve it — root cause was Redis connection pool exhaustion, fix was raising maxclients to 10000."
4. "Resolve it again." → tool returns the conditional-check error → Claude explains it's already resolved.
5. "Show me stats for checkout-service." → `incident_count`, `last_incident_at`.

11. Verification Checklist

- ☐ `uv run pytest` passes (moto-mocked DynamoDB).
- ☐ MCP Inspector (stdio) lists 7 tools, 1 resource, 1 prompt and each works against DynamoDB Local.
- ☐ `aws dynamodb query --table-name incident_memory --key-condition-expression "PK = :p" --expression-attribute-values '{":p":{"S":"SERVICE#checkout-service"}}'` shows items with correct PK/SK/GSI1 attributes.

- ☐ `aws dynamodb query --index-name GSI1 ...STATUS#open` returns only open incidents.
- ☐ Second `update_incident_status` on the same incident fails with *ConditionalCheckFailedException*.
- ☐ A note created with `ttl_days=0` (expires_at in the past) disappears within ~48 h (TTL is eventual).
- ☐ `curl` to `/mcp` without the token → 401; with token → initialize response.
- ☐ Claude Code connected over HTTPS runs the 5-step demo end to end.
- ☐ `sudo reboot` the EC2 → service comes back by itself (systemd + restart policy).
- ☐ CloudWatch Logs group `/incident-memory` shows request logs.
- ☐ The instance has **no** `~/.aws/credentials` file — `aws sts get-caller-identity` on the box shows the *assumed role*.

12. Costs, Cleanup & Security Checklist

Item	Cost
EC2 t3.micro / t2.micro	Free tier 750 h/month for first 12 months; afterwards ≈ \$8-9/month. Stop the instance when not demoing.
DynamoDB on-demand	Free tier 25 GB + 25 WCU/RCU-equivalent; a portfolio project costs ≈ \$0.
Elastic IP	Free while attached to a running instance; ≈ \$3.6/month if idle/unattached.
CloudWatch Logs	5 GB free; negligible.
Domain	≈ \$10/year (optional).

Set a **Billing alert** (Billing → Budgets → \$5/month) on day one.

Cleanup when finished

```
aws ec2 terminate-instances --instance-ids i-xxxx
aws ec2 release-address --allocation-id eipalloc-xxxx
aws dynamodb delete-table --table-name incident_memory
aws iam delete-access-key --user-name dev-admin --access-key-id AKIA... # rotate/delete
laptop keys
```

Security checklist

- ☐ MFA on root; root not used for daily work.
- ☐ No access keys on EC2 — instance role only; least-privilege policy scoped to one table + its GSI.
- ☐ Security group: 22 only from your IP; 8080 never public; app bound to 127.0.0.1.
- ☐ HTTPS everywhere; bearer token ≥ 32 random bytes; token not committed to git (`.gitignore` the nginx config or template it).
- ☐ `.env` files in `.gitignore`; `.env.example` committed instead.
- ☐ Dependencies pinned via `uv.lock`; Docker image uses slim base.

13. Resume Bullets & Interview Questions

Resume bullets (adapt the numbers to what you actually built)

- Designed and deployed **Incident Memory**, a remote **MCP server** (Python, FastMCP, Streamable HTTP) giving LLM agents searchable long-term memory of production incidents; 7 tools, 1 resource, 1 prompt.
- Modelled incident data with **DynamoDB single-table design** (composite keys, 1 GSI for cross-service status queries, TTL for expiring runbook notes) and enforced consistency with **conditional writes** and atomic counters.
- Deployed on **AWS EC2** with Docker, systemd, nginx TLS termination and bearer-token auth; eliminated static credentials by using an **IAM instance role** with a least-privilege policy; shipped logs to CloudWatch.
- Wrote unit tests with pytest + moto and validated the protocol surface with MCP Inspector; documented access patterns in a DESIGN.md.

Likely interview questions — and what to answer

Question	Key points
Why DynamoDB over PostgreSQL?	Predictable single-digit-ms latency at any scale, serverless/on-demand billing, no connection pooling on a tiny EC2. Trade-off: you must know access patterns up front; no ad-hoc joins.
Why single-table design?	One table = one set of capacity/limits; related items share a partition so one Query fetches incidents + notes + meta for a service.
Why a GSI? What does it cost?	Enables "all open incidents" without Scan. Costs extra write units (each write is replicated to the index) and storage; eventually consistent.
What is a hot partition and could you have one?	Too much traffic to one PK. A very noisy service could become hot; mitigation: write-sharding the PK (<code>SERVICE#x#0..N</code>) or time-bucketing.
How does the conditional write prevent race conditions?	DynamoDB evaluates the condition atomically on the server; two concurrent resolves → exactly one succeeds.
How does boto3 get credentials on EC2?	Credential provider chain → Instance Metadata Service (IMDSv2) returns temporary STS credentials for the attached role, rotated automatically.
Difference between MCP tools, resources and prompts?	Tools: model-invoked actions. Resources: app-controlled data by URI. Prompts: user-selected templates.
stdio vs Streamable HTTP?	stdio = local subprocess, simplest; Streamable HTTP = network service, one endpoint, supports SSE streaming, needs auth/TLS.
How would you scale this?	Stateless server → run N containers behind an ALB; DynamoDB scales automatically; move auth to OAuth 2.1 as the MCP spec recommends; add DynamoDB Streams → Lambda for analytics.
What would you do differently in production?	OAuth instead of a static token, Terraform/CDK for infra, CI/CD pipeline, structured logging + metrics, rate limiting, WAF.

Suggested timeline (part-time)

Week	Deliverable
1	Phase 1 + 2: local server with all tools against DynamoDB Local; tests passing; DESIGN.md
2	Phase 3: AWS account hardening, IAM, real table; server runs locally against real AWS
3	Phase 4: EC2 deploy, Docker, systemd, nginx, HTTPS, token auth
4	Phase 5: Claude connected; README with diagram + GIF; GitHub Actions running pytest; publish on LinkedIn

End of guide.